

# R12 実験プログラム仕様書および出力結果の説明

## 1 プログラムの目的

本プログラム `r12_experiment.py` は、ビット列に含まれる規則性をニューラルネットワークがどの程度学習できるかを調べるための実験プログラムである。直前のビット列を入力として次の 1 ビットを予測する二値分類問題を設定し、以下の 3 種類の生成器を比較する。

1. 4 ビット LFSR
2. 非線形フィルタ付き 4 ビット LFSR
3. Python 標準ライブラリ `secrets` による乱数列

各生成器について同じ長さ・同じ個数のビット列を生成し、同一構造の多層パーセプトロンを用いて次ビット予測を行う。これにより、生成方式の違いが予測性能に与える影響を比較する。

## 2 対象とするビット列生成器

### 2.1 4 ビット LFSR

4 ビット LFSR では、初期状態として 0000 以外の 4 ビットをランダムに選ぶ。漸化式は原始多項式

$$x^4 + x + 1$$

に対応するものとし、次ビットを

$$s_{t+4} = s_{t+1} \oplus s_t$$

で生成する。ここで、 $\oplus$  は排他的論理和を表す。

この生成器では、直前 4 ビット

$$(s_t, s_{t+1}, s_{t+2}, s_{t+3})$$

が分かれば、次ビット  $s_{t+4}$  は一意に決まる。そのため、ニューラルネットワークが対応関係を十分に学習できれば、高い予測精度が得られると考えられる。

### 2.2 非線形フィルタ付き 4 ビット LFSR

非線形フィルタ付き LFSR では、内部状態の更新には通常の 4 ビット LFSR と同じ漸化式を用いる。一方、外部へ出力するビットには次の非線形関数を用いる。

$$z_t = s_{t,0} \oplus (s_{t,1} \wedge s_{t,2}) \oplus s_{t,3}.$$

ここで、 $\wedge$  は論理積を表す。内部状態は線形に更新されるが、出力には非線形処理が加えられるため、通常の LFSR よりも入力ビット列と次ビットの関係は複雑になる。

## 2.3 secrets による乱数列

secrets による乱数列では、`secrets.randbits(1)` を用いて各ビットを独立に生成する。この生成器では、過去の 4 ビットと次の 1 ビットの間に利用可能な決定的規則は存在しないと考えられる。したがって、予測精度は多数派予測やランダム予測と同程度になることが期待される。

## 3 データセットの仕様

各生成器から、長さ 19 ビットの系列を 200 本生成する。1 本の系列を

$$z_0, z_1, \dots, z_{18}$$

とすると、入力と教師ラベルは次のように作成する。

$$x_t = (z_t, z_{t+1}, z_{t+2}, z_{t+3}), \quad y_t = z_{t+4}.$$

長さ 19 ビットの系列からは、 $19 - 4 = 15$  個の教師付きサンプルが得られる。系列の分割は、サンプル作成前に行う。これにより、時間的に隣接したサンプルが学習用とテスト用にまたがることを避けている。

表 1 データ分割

| 用途   | 系列数 | サンプル数 |
|------|-----|-------|
| 学習用  | 120 | 1800  |
| 検証用  | 40  | 600   |
| テスト用 | 40  | 600   |

## 4 ニューラルネットワークの仕様

本実験では、TensorFlow/Keras を用いて小規模な多層パーセプトロンを実装した。ネットワーク構造は以下の通りである。

表 2 モデル構造

| 層    | ユニット数 | 活性化関数   |
|------|-------|---------|
| 入力層  | 4     | —       |
| 全結合層 | 8     | ReLU    |
| 全結合層 | 4     | ReLU    |
| 出力層  | 1     | Sigmoid |

出力層は、次ビットが 1 である確率を出力する。出力値が 0.5 以上の場合を 1, 0.5 未満の場合を 0 として予測ラベルに変換する。

入力ビット数を 4 としたとき、学習可能パラメータ数は

$$(4 \times 8 + 8) + (8 \times 4 + 4) + (4 \times 1 + 1) = 81$$

である。

## 5 学習条件

主な学習条件を表 3 に示す。各生成器について、乱数シードを変えて 3 回学習を行い、評価値の平均と標準偏差を求める。

表 3 学習条件

| 項目             | 設定値                   |
|----------------|-----------------------|
| 損失関数           | binary crossentropy   |
| 最適化手法          | Adam                  |
| 学習率            | 0.001                 |
| バッチサイズ         | 16                    |
| 最大エポック数        | 50                    |
| Early Stopping | 検証損失を監視, patience = 8 |
| 実行回数           | 各生成器 3 回              |

## 6 実行方法

プログラムは以下のコマンドで実行する。

```
python r12_experiment.py
```

出力先は既定で `r12_results` ディレクトリである。主なオプションとして、入力ビット数、系列数、エポック数、実行回数などをコマンドライン引数から変更できる。

```
python r12_experiment.py --runs 3 --epochs 50 --input-window 4
```

## 7 出力ファイルの説明

実行後、`r12_results` ディレクトリに以下のファイルが保存される。

表 4 出力ファイル

| ファイル名                         | 内容                       |
|-------------------------------|--------------------------|
| results.csv                   | 各生成器・各実行回の詳細な評価結果        |
| summary.csv                   | 各生成器について 3 回実行した平均値と標準偏差 |
| test_accuracy_summary.png     | テスト正解率の比較グラフ             |
| balanced_accuracy_summary.png | Balanced Accuracy の比較グラフ |
| learning_curves.png           | 学習損失および検証損失の推移           |

## 7.1 results.csv

results.csv には、各実行ごとの詳細な評価値が記録される。主な列の意味は以下の通りである。

表 5 results.csv の主な列

| 列名                         | 意味                       |
|----------------------------|--------------------------|
| generator                  | 使用した生成器                  |
| run                        | 実行番号                     |
| seed                       | 乱数シード                    |
| train_samples              | 学習サンプル数                  |
| val_samples                | 検証サンプル数                  |
| test_samples               | テストサンプル数                 |
| epochs_ran                 | 実際に学習したエポック数             |
| train_loss                 | 学習データに対する損失              |
| val_loss                   | 検証データに対する損失              |
| test_loss                  | テストデータに対する損失             |
| accuracy                   | テストデータに対する正解率            |
| precision                  | 適合率                      |
| recall                     | 再現率                      |
| f1                         | F 値                      |
| balanced_accuracy          | クラス不均衡を補正した正解率           |
| tn, fp, fn, tp             | 混同行列の各要素                 |
| baseline_accuracy          | 多数派予測の正解率                |
| baseline_balanced_accuracy | 多数派予測の Balanced Accuracy |

## 7.2 summary.csv

summary.csv には、各生成器について 3 回実行した結果の平均値と標準偏差が記録される。たとえば、accuracy\_mean はテスト正解率の平均、accuracy\_std はその標準偏差を表す。

## 8 出力結果の解釈

今回の実行結果の概要を表 6 に示す。

表 6 実験結果の概要

| 生成器            | Accuracy | Balanced Accuracy | Baseline Accuracy |
|----------------|----------|-------------------|-------------------|
| 4 ビット LFSR     | 1.000    | 1.000             | 0.533             |
| 非線形フィルタ付き LFSR | 0.733    | 0.747             | 0.533             |
| <b>secrets</b> | 0.508    | 0.507             | 0.500             |

4 ビット LFSR では、Accuracy および Balanced Accuracy がともに 1.000 となった。これは、直前 4 ビットから次ビットが線形漸化式により一意に決まるためである。ニューラルネットワークは、この対応関係を完全に学習できたと考えられる。

非線形フィルタ付き LFSR では、通常の LFSR より予測精度は低下したものの、多数派ベースラインを明確に上回った。したがって、非線形フィルタを通した出力列にも、次ビット予測に利用可能な規則性または偏りが残っていると考えられる。ただし、標準偏差が比較的大きいため、初期値や学習のばらつきの影響も受けている。

**secrets** による乱数列では、Accuracy および Balanced Accuracy がほぼ 0.5 となった。これは、直前 4 ビットから次の 1 ビットを予測する有効な規則性をニューラルネットワークが学習できなかったことを示す。また、損失も二値分類におけるランダム予測に対応する 0.693 付近にとどまっている。

## 9 グラフの説明

### 9.1 テスト正解率

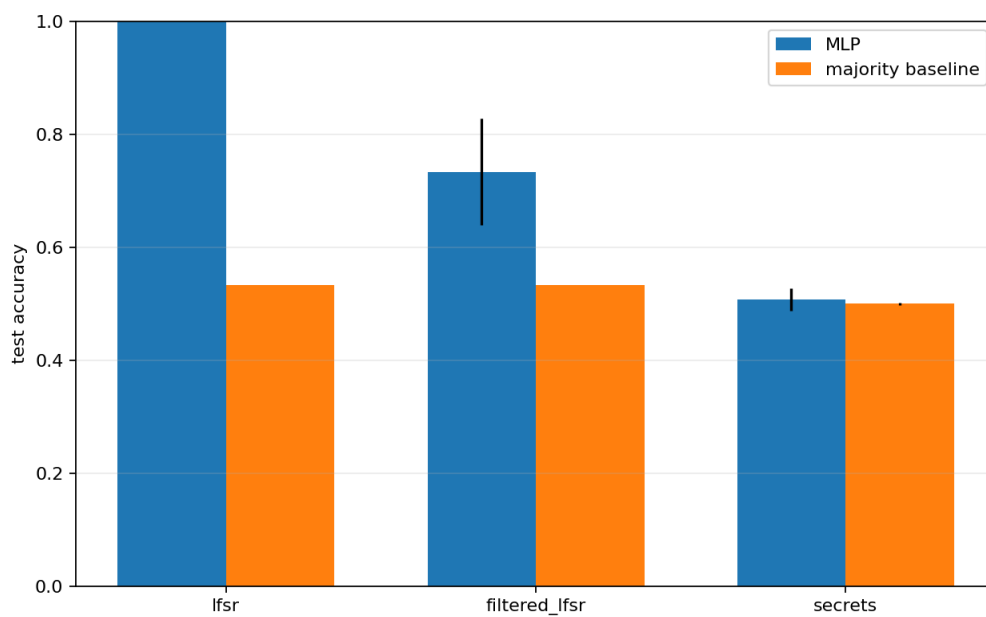


図1 テスト正解率の比較

この図は、各生成器に対する MLP のテスト正解率と、多数派ベースラインの正解率を比較したものである。LFSR では MLP が完全な予測を達成している。非線形フィルタ付き LFSR では MLP がベースラインを上回る一方、secrets では MLP とベースラインの差がほとんどない。

## 9.2 Balanced Accuracy

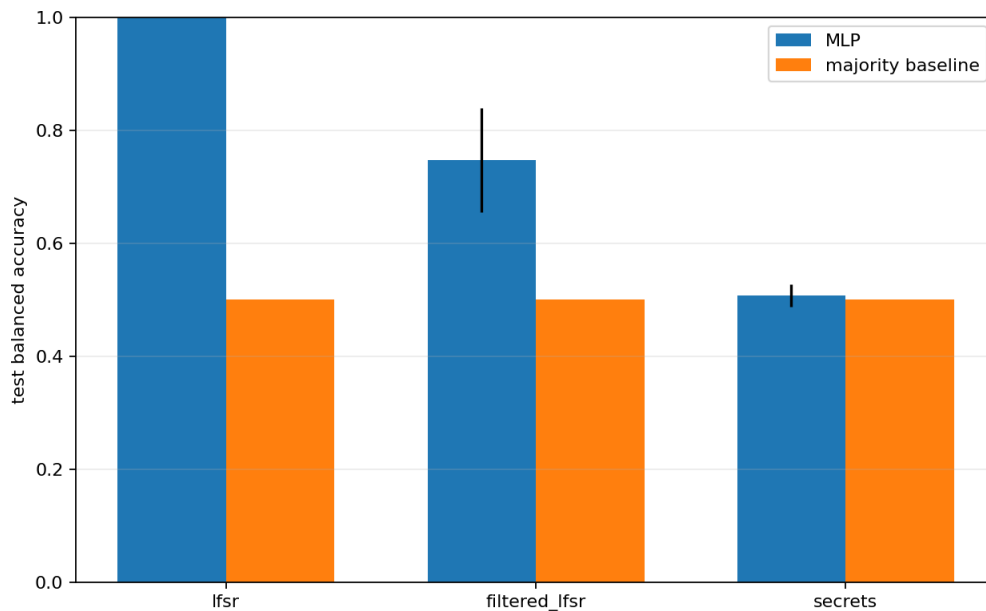


図2 Balanced Accuracy の比較

Balanced Accuracy は、0 と 1 の出現数に偏りがある場合でも性能を評価しやすい指標である。多数派ベースラインは常に多いクラスを予測するため、通常の正解率は 0.5 を少し超える場合がある。しかし、Balanced Accuracy では 0.5 となり、実質的には有効な予測を行っていないことが分かる。

## 9.3 学習曲線

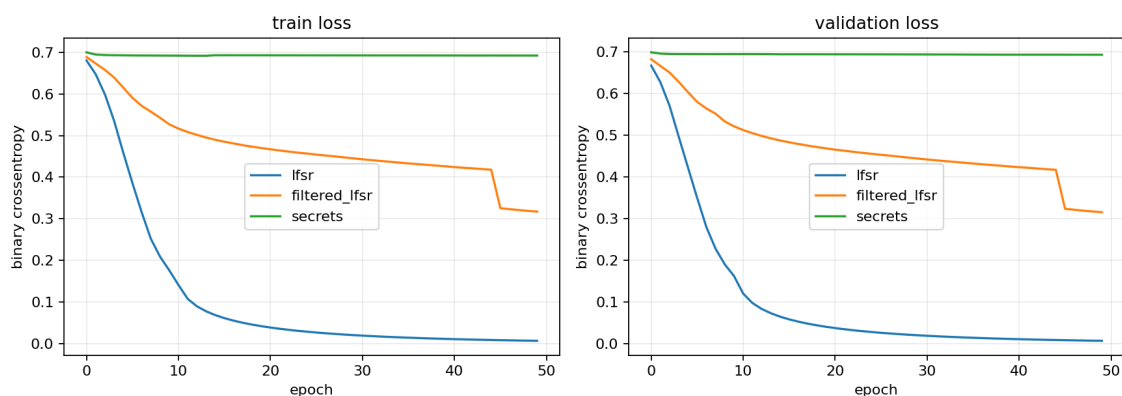


図3 学習損失と検証損失の推移

学習曲線を見ると、LFSR では学習損失・検証損失が急速に 0 へ近づいている。これは、学習データだけでなく未知の検証データに対しても同じ規則が適用できていることを示す。

非線形フィルタ付き LFSR では、損失は減少するが 0 までは到達しない。これは、観測される直前 4 ビットだけから次ビットを完全に決定することが難しいためである。

`secrets` では、損失がほぼ 0.693 付近に留まる。これは、二値分類において予測確率がほぼ 0.5 に近い状態であり、過去のビットから次ビットを予測する情報が得られていないことを意味する。

## 10 まとめ

本実験により、4 ビット LFSR のような小規模で決定的な生成器では、ニューラルネットワークが次ビットの規則性を完全に学習できることが確認された。非線形フィルタ付き LFSR では、予測は難しくなるものの、出力列にはまだ学習可能な構造が残っていた。一方、`secrets` による乱数列では、予測精度はほぼ 0.5 に留まり、直前 4 ビットから次ビットを予測する有効な規則性は確認できなかった。

ただし、本実験は 4 ビットという非常に小規模な設定であり、LFSR の予測成功をもって一般的な暗号乱数の安全性を評価することはできない。本実験は、ニューラルネットワークがビット列中の規則性を学習できるかを確認するための基礎実験として位置付けられる。